

StockSense AI — Technical Architecture & Review 2 Guide

AI-Powered Supply Chain & Inventory Intelligence System for Indian Kirana Stores Review 2 Readiness
Document | March 2026

Table of Contents

- [1. The Problem & Our Solution](#)
- [2. Tech Stack — What, Where & Why](#)
- [3. System Architecture — The Big Picture](#)
- [4. Database Layer — Supabase / PostgreSQL](#)
- [5. Backend Layer — n8n Workflow Engine](#)
- [6. Frontend Layer — React SPA](#)
- [7. API & Integration Layer](#)
- [8. The Daily Intelligence Pipeline — End to End](#)
- [9. AI / LLM Integration — Groq + Llama 3.1](#)
- [10. External Integrations — Twilio & WhatsApp](#)
- [11. Realtime Architecture — Live Updates](#)
- [12. Security & Authentication](#)
- [13. Key Design Decisions & Trade-offs](#)
- [14. Demo Script — Review 2 Walkthrough](#)

1. The Problem & Our Solution

Problem: Small Indian retail (kirana) stores manage 200–2000 SKUs manually. They have no data-driven way to know *when* a product will run out, *how much* to reorder, or *which supplier* gives the best value. This leads to frequent stockouts (lost sales) and overstocking (dead capital).

Our Solution — StockSense AI: A web-based inventory intelligence system that:

- **Predicts** when each product will go out of stock (days-to-stockout projection)
- **Classifies** products into risk tiers (HIGH / MEDIUM / LOW / OOS)
- **Generates AI-powered reorder recommendations** using an LLM that considers supplier reliability, pricing, and delivery speed
- **Sends WhatsApp alerts** to the store owner about critical items
- **Tracks inventory health** over time with a daily health score

The entire pipeline runs automatically every day at 8 AM, or on-demand via a single button click.

2. Tech Stack — What, Where & Why

Frontend

Technology	Version	Where Used	Why Chosen
React	19	Entire UI (SPA)	Component-based architecture allows rapid UI development. React 19 brings performance improvements with the new compiler. Industry standard — easy to find docs and support.

Vite	8	Build tool & dev server	10–100× faster than Webpack during development. Native ES module support gives instant hot module replacement (HMR). Built-in proxy server eliminates CORS issues during dev.
Tailwind CSS	v4	All styling	Utility-first CSS eliminates context switching. v4 uses the new <code>@theme</code> directive — no config file needed. Produces tiny CSS bundles in production via tree-shaking.
React Router	7	Client-side routing	Industry standard for React SPAs. Supports nested routes, layout routes, and protected routes out of the box.
Recharts	—	Dashboard health chart	Lightweight React-native charting library. Renders SVG-based charts with smooth animations. Easy to customize with Tailwind.
Lucide React	—	All icons	Modern, consistent icon set with tree-shakeable imports — only icons you use get bundled.
React Hot Toast	—	Notifications	Lightweight, accessible toast notifications with promise support. Zero-config, beautiful defaults.

Backend (Workflow Engine)

Technology	Where Used	Why Chosen
n8n (self-hosted)	All backend logic (8 workflows)	Visual workflow builder — no traditional server code needed. Supports 400+ integrations. Easy to modify business logic without redeploying. Perfect for a project that needs rapid iteration.
Groq API	WF-05 (AI Reorder Intelligence)	Fastest LLM inference API available (~100ms latency). Free tier generous enough for demo/dev. Hosts Llama 3.1 8B — open-source model, no vendor lock-in.
Twilio	WF-07 (WhatsApp Alerts)	Industry-standard messaging API. WhatsApp Business API integration is production-ready. Sandbox mode available for free testing.
ngrok	Dev tunneling for n8n	Exposes local n8n instance to the internet for webhook testing. Gives a stable HTTPS URL that the frontend can call. Required because n8n runs on <code>localhost:5678</code> .

Database & Auth

Technology	Where Used	Why Chosen
Supabase	Database, Auth, Realtime	Open-source Firebase alternative built on PostgreSQL. Provides: (1) Managed Postgres with a REST API — no backend needed for CRUD. (2) Built-in auth with email/password. (3) Realtime subscriptions via WebSockets — UI updates instantly. (4) Row Level Security (RLS) for multi-tenancy. One platform replaces 3–4 separate services.
PostgreSQL	Underlying DB engine	The world's most advanced open-source relational DB. Supports JSON, full-text search, and extensions like <code>pgcrypto</code> . Battle-tested for supply chain data.

Why This Architecture? (The "No Traditional Backend" Philosophy)

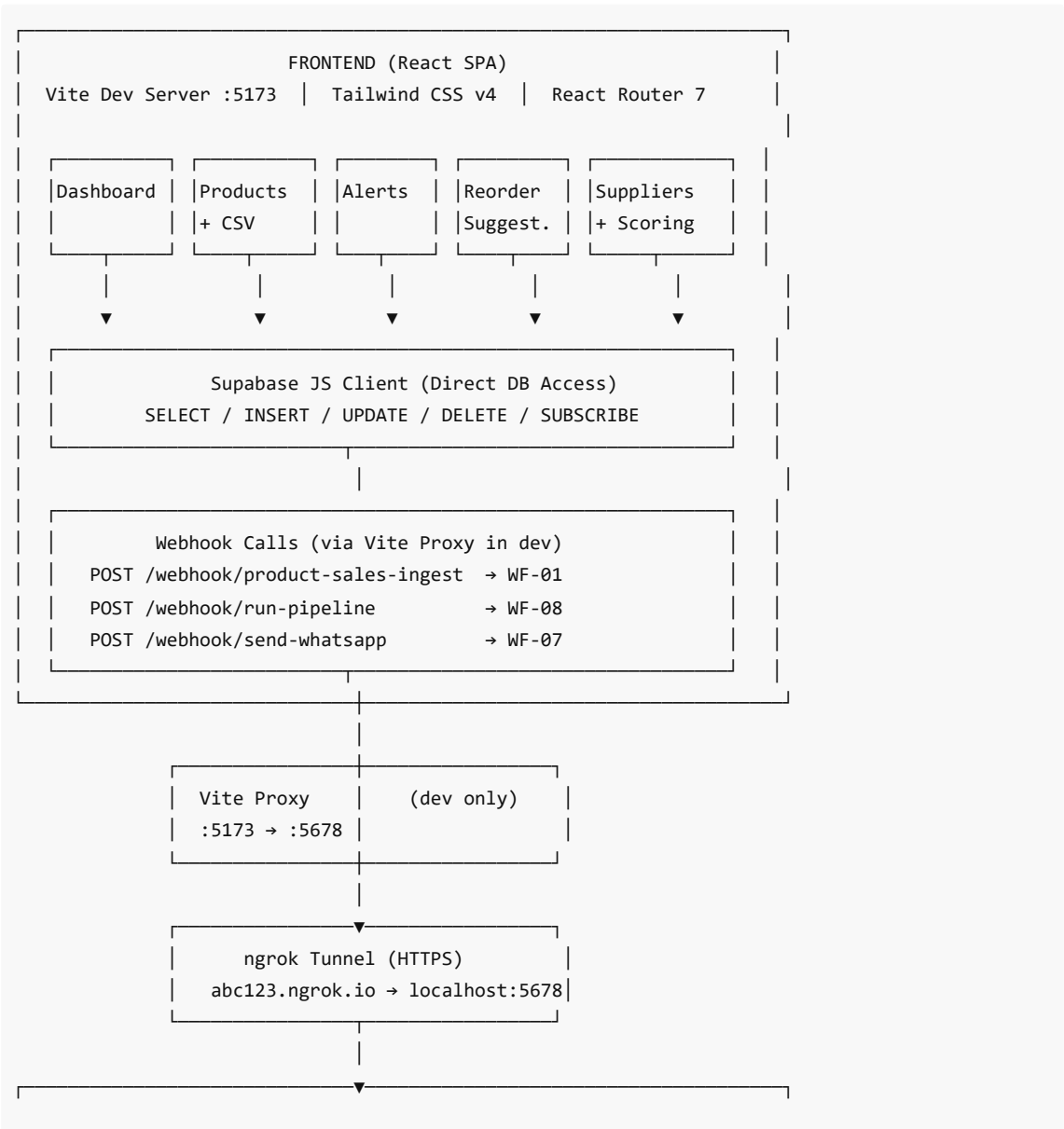
Traditional apps have: Frontend → REST API (Express/Django) → Database

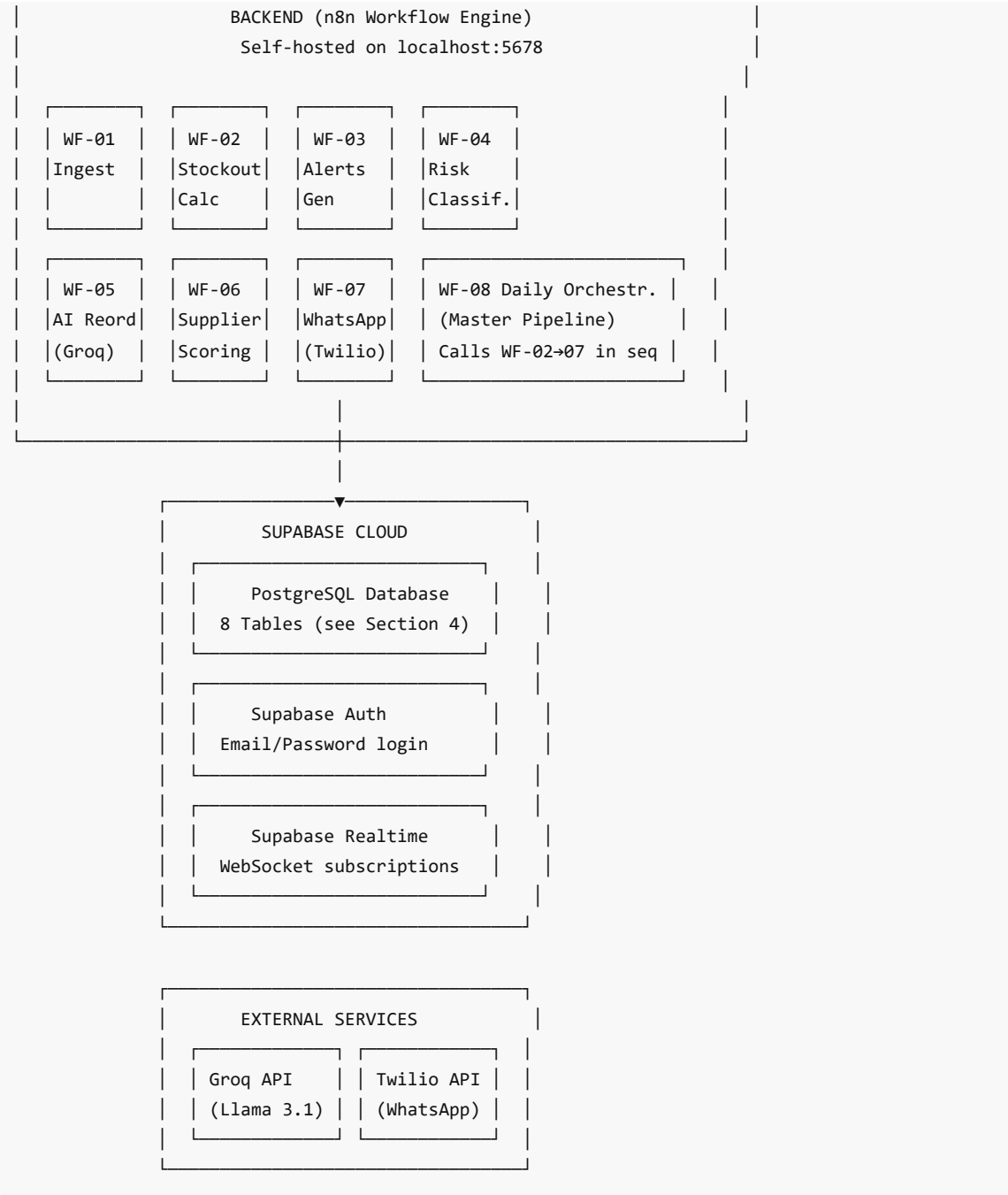
Our approach: Frontend → Supabase (direct DB access) + n8n (async workflows)

Why?

- **Supabase replaces the CRUD API layer** — the React frontend calls Supabase directly for all reads/writes. No Express server, no Django, no API boilerplate.
- **n8n replaces the business logic layer** — all complex computations (risk classification, stockout calculation, AI recommendations) run as visual workflows triggered by webhooks or schedules.
- **Result:** Zero server-side code to maintain. The entire "backend" is configuration (n8n JSON exports + Supabase schema).

3. System Architecture — The Big Picture

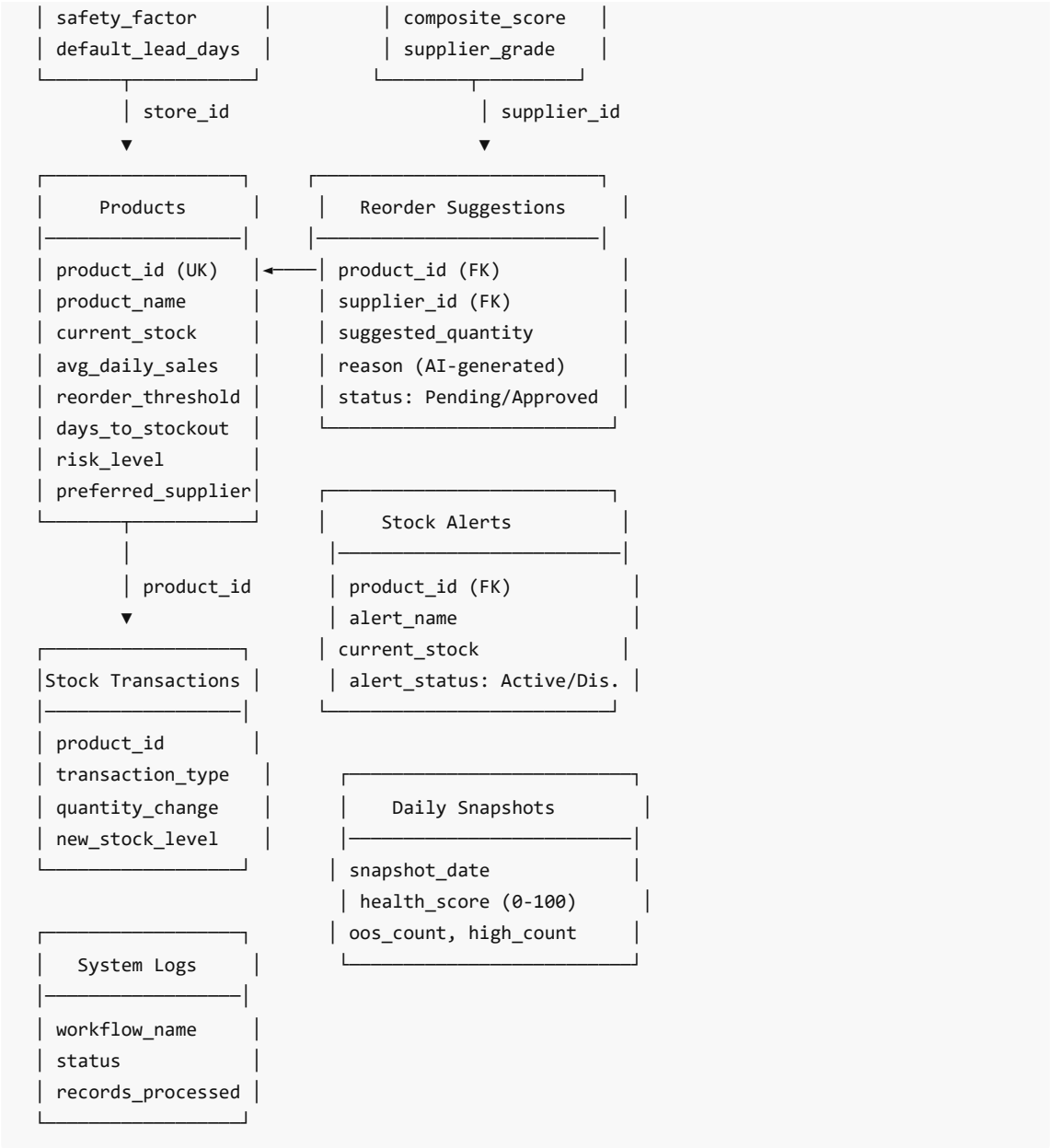




4. Database Layer — Supabase / PostgreSQL

Entity-Relationship Overview

Store Profiles	Suppliers
id (PK, UUID)	supplier_id (UK)
user_id → auth	supplier_name
store_name	reliability_score
whatsapp_numbers	price_score



The 8 Tables — Quick Reference

#	Table	Purpose	Updated By
1	Store Profiles	One row per registered store (owner info, WhatsApp numbers, safety factor)	Signup, Onboarding, Settings
2	Products	Master product catalog — stock levels, sales velocity, risk classification, stockout projections	WF-01 (ingest), WF-02 (stockout), WF-04 (risk), WF-08 (simulation)
3	Suppliers	Supplier directory with multi-criteria scoring (reliability, price, delivery speed)	Manual + WF-06 (scoring)

4	Stock Alerts	Low stock alerts when <code>current_stock ≤ reorder_threshold</code>	WF-03 (full refresh each pipeline run)
5	Reorder Suggestions	AI-generated reorder recommendations with supplier matching and quantity	WF-05 (AI via Groq/Llama)
6	Stock Transactions	Immutable audit ledger of all stock changes (sales, restocks, adjustments)	WF-08 (daily depletion), Quick Update page
7	Daily Snapshots	Daily health score snapshots powering the trend chart	WF-08 (end of pipeline)
8	System Logs	Workflow execution logs for monitoring and debugging	Every workflow logs its status here

Supabase Realtime

These tables have **Realtime enabled** so the frontend updates instantly without page refresh:

- Products, Stock Alerts, Reorder Suggestions, Daily Snapshots, Suppliers

How it works: When n8n updates a row in the `Products` table, Supabase broadcasts the change over WebSocket → React's `useEffect` subscription fires → `fetchData()` re-runs → UI updates in real-time.

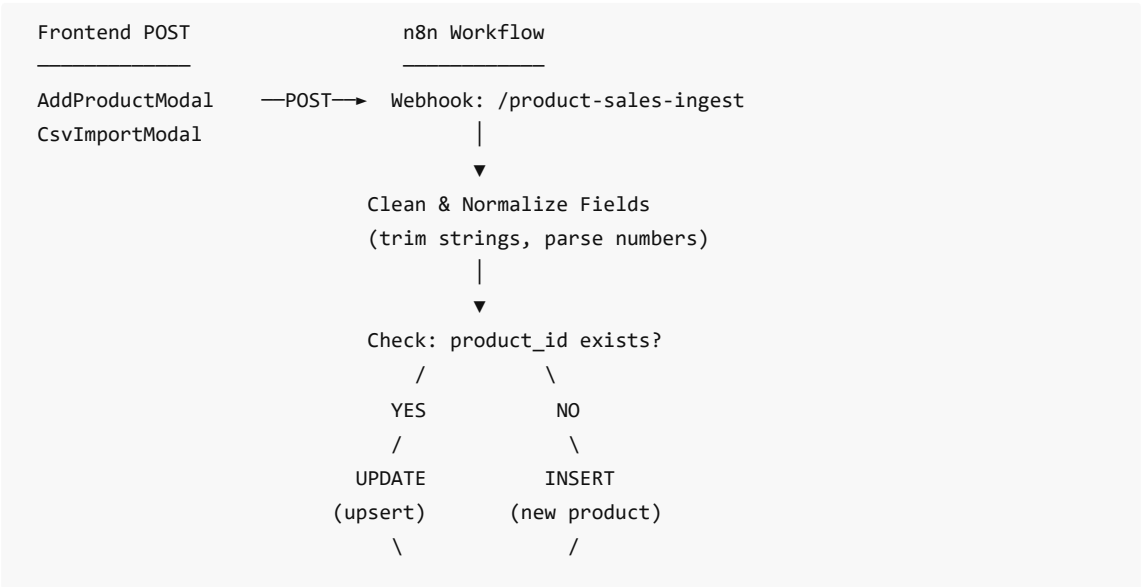
5. Backend Layer — n8n Workflow Engine

What is n8n?

n8n is a **visual workflow automation platform** (self-hosted). Think of it as "Zapier but you host it yourself." Each workflow is a chain of nodes (Supabase queries, code blocks, HTTP requests, IF conditions) connected visually. The workflows are exported as JSON files stored in `backend/`.

The 8 Workflows — Detailed Breakdown

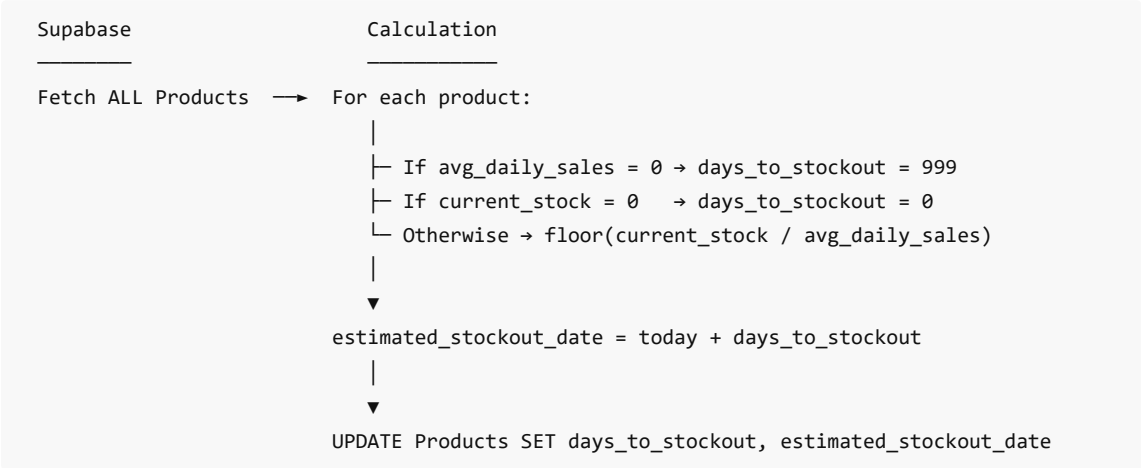
WF-01: Product & Sales Data Ingestion



\ /
▼ ▼
Return { action: 'updated' | 'created' }

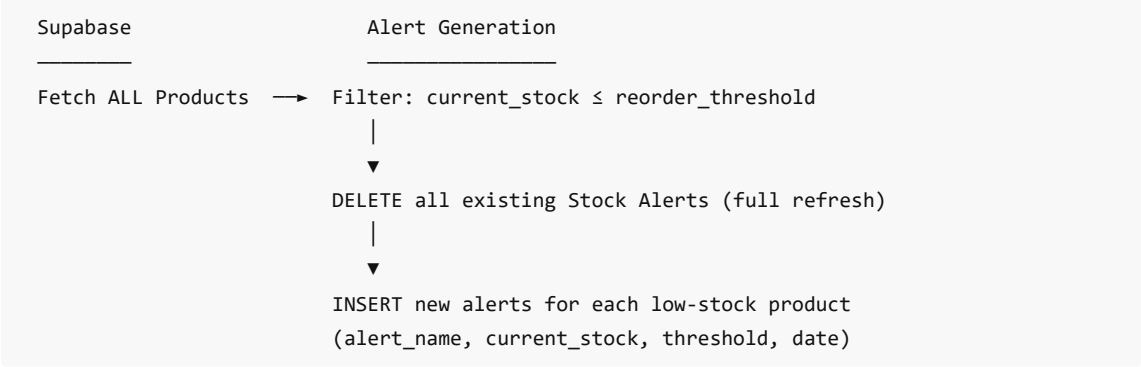
Trigger: POST webhook `/product-sales-ingest` **Called by:** React frontend (AddProductModal, CsvImportModal)
Purpose: Receives product/sales data, cleans it, and upserts into the Products table.

WF-02: Stockout Date Calculator



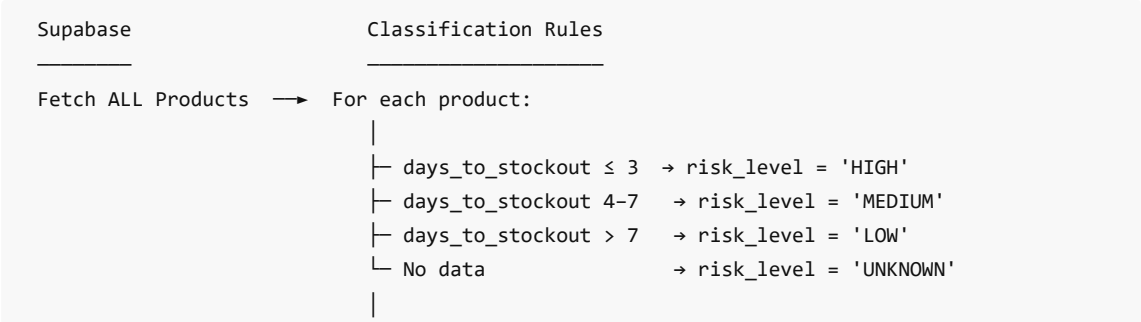
Trigger: Sub-workflow call from WF-08 **Purpose:** Projects when each product will run out of stock.

WF-03: Inventory Processing & Stock Alerts



Strategy: Full-refresh (delete-and-recreate). This ensures alerts always reflect the *current* state. No stale alerts persist.

WF-04: Stock Risk Classification



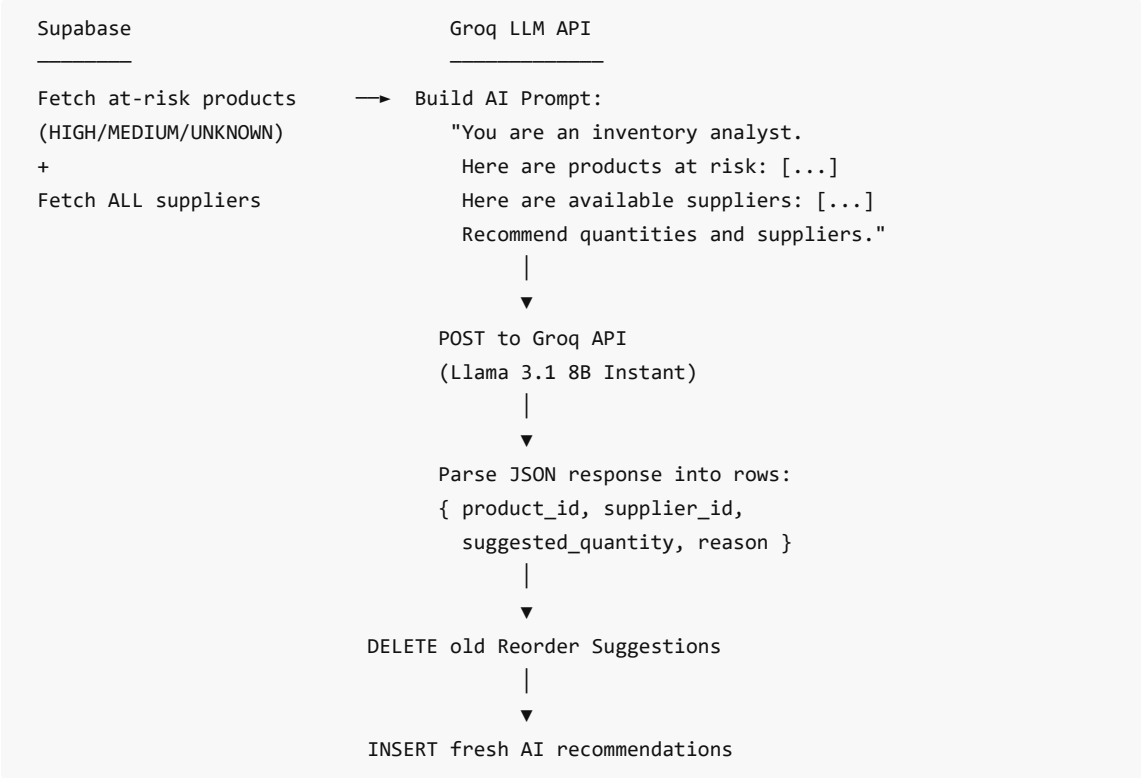
▼

UPDATE Products SET risk_level

Depends on: WF-02 (needs `days_to_stockout` calculated first)

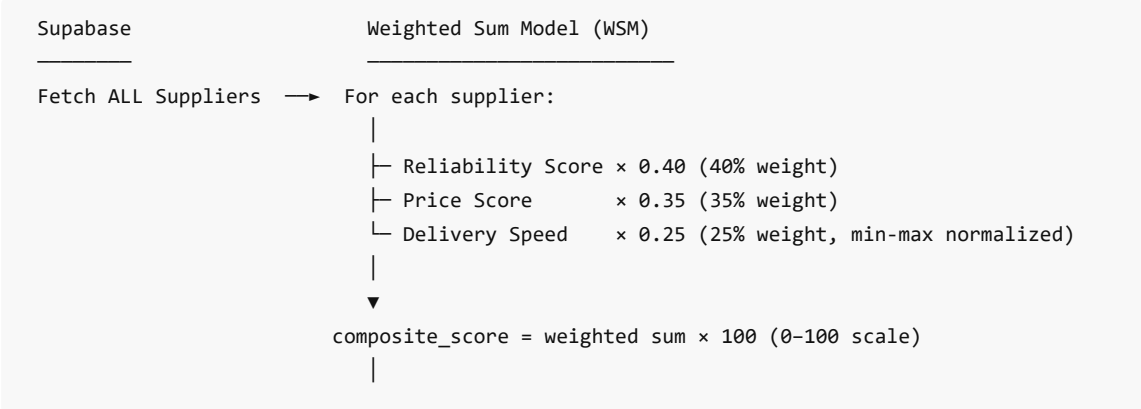
Note (Frontend OOS Override): In the React UI, the `RiskBadge` component has an additional rule: if `current_stock = 0`, it always displays **OOS** regardless of the database `risk_level`. This gives clearer visual feedback for completely empty stock.

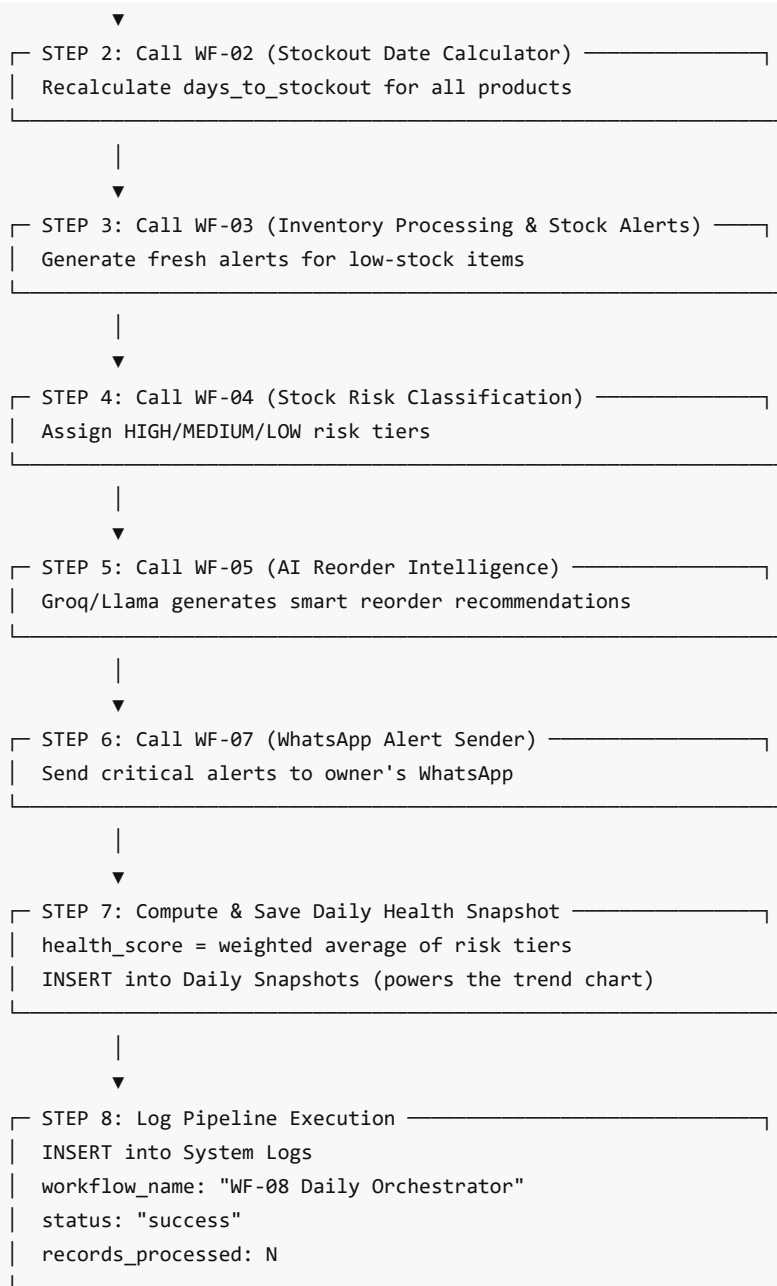
WF-05: AI Reorder Intelligence ★ (The AI Core)



AI Model: Groq — Llama 3.1 8B Instant **Why Groq?** Fastest inference (~100ms), free tier, runs Meta's open-source Llama model. **What the AI considers:** Current stock, avg daily sales, days to stockout, supplier reliability scores, price competitiveness, delivery speed, store's safety factor.

WF-06: Supplier Scoring

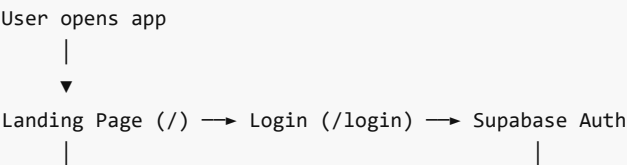


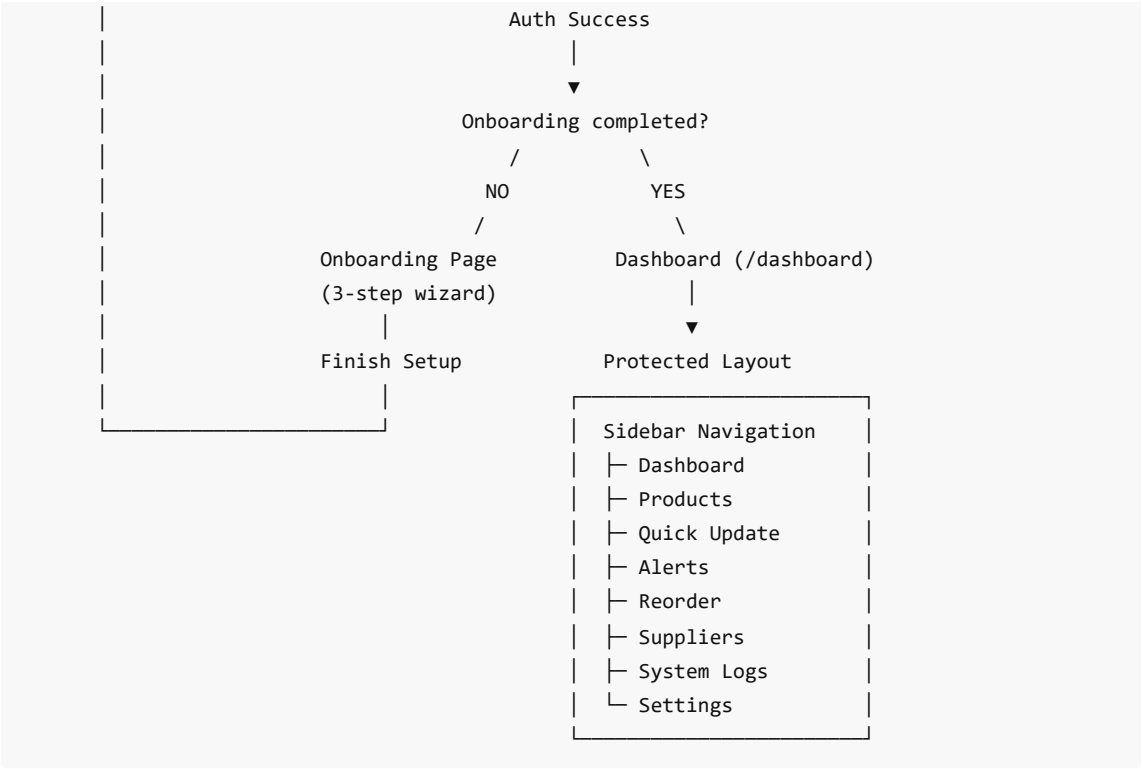


Key Intelligence: Products that were manually updated by the store owner within the last 24 hours are **skipped** during stock simulation. This prevents the system from overriding a real physical stock count.

6. Frontend Layer — React SPA

Application Flow





Page-by-Page Breakdown

Page	Route	Key Features
Dashboard	/dashboard	Health score (0–100), 4 KPI cards (alerts, reorders, high-risk, OOS), "Needs Attention" table, health trend chart (Recharts), "Run Pipeline" button, "Send WhatsApp" button
Products	/products	Full product table with inline editing, CSV import modal, risk badges (OOS/HIGH/MED/LOW), search & filter, add/delete products
Quick Update	/quick-update	Fast stock count updates — designed for daily use. Slider + input for quickly adjusting stock levels.
Alerts	/alerts	Active/Dismissed alert cards with stock level bars. Dismiss one or all. Realtime updates.
Reorder	/reorder	AI-generated reorder suggestion cards with AI badge (🔮). Approve/Dismiss actions. WhatsApp supplier button for approved orders. Shows reason, quantity, estimated cost.
Suppliers	/suppliers	Supplier table with grade badges (A/B/C/D), composite score bars, inline editing, add/delete. Performance visualization.
Settings	/settings	Store profile form, pipeline preferences (safety factor, lead days, alert time), workflow trigger buttons.
System Logs	/logs	Workflow execution history — shows which workflows ran, when, status, and records processed.

State Management Strategy

- **Global:** `AuthContext` (React Context) — provides `session` , `user` , `storeProfile` , `signIn()` , `signUp()` , `signOut()`
- **Page-level:** `useState` hooks — no Redux or Zustand. Each page fetches its own data from Supabase on mount.
- **Why no Redux?** The app is a dashboard — each page has its own data scope. No complex cross-page state sharing needed.

7. API & Integration Layer

The 3 Webhook Endpoints

Endpoint	Method	Called By	n8n Workflow	Purpose
<code>/webhook/product-sales-ingest</code>	POST	AddProductModal, CsvImportModal	WF-01	Ingest a single product row
<code>/webhook/run-pipeline</code>	POST	Dashboard "Run Pipeline" button	WF-08	Trigger the full daily intelligence pipeline
<code>/webhook/send-whatsapp</code>	POST	Dashboard "Send WhatsApp" button	WF-07	Send WhatsApp alert to store owner

How the Proxy Works (Development)

Browser (:5173)	Vite Dev Server	n8n (:5678)
POST <code>/webhook/run-pipeline</code>	→ Vite sees <code>/webhook/*</code> → Proxy to <code>localhost:5678</code> (<code>vite.config.js</code> proxy)	→ n8n receives the call

Why? In development, n8n runs on `localhost:5678` and the React app on `localhost:5173` . Cross-origin requests would be blocked by CORS. The Vite proxy makes the browser think it's talking to the same origin.

Production: The frontend calls n8n via its ngrok URL directly (set in `VITE_N8N_BASE_URL` env var).

Direct Supabase Access (No REST API)

For all data queries (Products, Alerts, Suppliers, etc.), the React frontend calls Supabase's JavaScript client **directly** — no intermediate REST API:

```
// Example: Fetch all products
const { data, error } = await supabase.from('Products').select('*')

// Example: Update stock
await supabase.from('Products').update({ current_stock: 50 }).eq('product_id', 'MILK_01')
```

This eliminates the need for a traditional Express/Django backend for CRUD operations.

8. The Daily Intelligence Pipeline — End to End

Data Flow: From CSV to WhatsApp Alert

Here's the complete journey of data through the system:



Health Score Calculation

For each product:

- LOW risk → scores 100 points
- MEDIUM risk → scores 70 points
- HIGH risk → scores 30 points
- OOS (stock=0) → scores 5 points

health_score = average of all product scores (0-100 scale)

Example: 10 products → 3 LOW (300) + 2 MEDIUM (140) + 3 HIGH (90) + 2 OOS (10) = 540 / 10 = **54**

9. AI / LLM Integration — Groq + Llama 3.1

How it Works

1. **WF-05** fetches all at-risk products (HIGH/MEDIUM/UNKNOWN risk level) and all suppliers
2. Builds a **detailed prompt** with full context:
 - Product name, stock, avg daily sales, days to stockout, reorder threshold
 - Supplier name, reliability score, price score, delivery time, categories
 - Store's safety factor (buffer multiplier)
3. Sends to **Groq API** (model: `llama-3.1-8b-instant`)
4. LLM returns **structured JSON** with recommendations:

```
[
  {
    "product_id": "MILK_01",
    "supplier_id": "SUP_DAIRY",
    "suggested_quantity": 50,
    "reason": "Critical stock: 0 units remaining with 12 units/day sales velocity.
Amul Dairy Corp has 8.5/10 reliability and 2-day delivery."
  }
]
```

5. WF-05 parses and inserts into `Reorder Suggestions` table

Why Groq over OpenAI?

Factor	Groq	OpenAI
Speed	~100ms inference	~1-3 seconds
Cost	Free tier (30 req/min)	Paid (\$0.15/1M tokens)
Model	Llama 3.1 8B (open-source)	GPT-4 (proprietary)
Vendor Lock-in	None — can switch to any Llama host	Locked to OpenAI API

10. External Integrations — Twilio & WhatsApp

WhatsApp Alert Flow

```
WF-07 triggers
|
▼
Fetch active Stock Alerts from Supabase
|
▼
Filter: Only alerts NOT sent in last 24 hours (anti-spam)
|
▼
```

```
Sort by urgency (lowest stock first)
  |
  ▼
Build message body (product list with stock levels)
  |
  ▼
POST to Twilio REST API:
  URL: https://api.twilio.com/2010-04-01/Accounts/{SID}/Messages
  From: whatsapp:+14155238886 (Twilio Sandbox number)
  To:   whatsapp:+91{owner_number}
  Body: " 🚨 StockSense AI Alert! ..."
  |
  ▼
Update last_alerted_at timestamp in Stock Alerts table
```

Twilio Sandbox vs Production

- **Demo:** Uses Twilio Sandbox (free, number must opt-in by sending "join " to sandbox number)
- **Production:** Would use a WhatsApp Business API number (paid, no opt-in required)

Reorder Page — WhatsApp to Supplier

The Reorder page has a separate WhatsApp feature for **contacting suppliers directly**:

- User approves an AI suggestion → clicks "WhatsApp Supplier"
- Opens `wa.me/{phone}?text={pre-filled order message}` in a new tab
- This uses the **WhatsApp Click-to-Chat API** (no Twilio needed — free, direct)

11. Realtime Architecture — Live Updates

How Supabase Realtime Works

```
n8n updates Products table
  |
  ▼
PostgreSQL publishes change event
(via ALTER PUBLICATION supabase_realtime ADD TABLE "Products")
  |
  ▼
Supabase Realtime server broadcasts via WebSocket
  |
  ▼
React useEffect subscription fires:
  |
  ▼
fetchData() re-runs → setState() → UI re-renders
```

Subscriptions in Dashboard.jsx

```
const channels = [
  supabase.channel('dash-products')
```

```

    .on('postgres_changes', { event: '*', schema: 'public', table: 'Products' }, () =>
    fetchData())
    .subscribe(),
    supabase.channel('dash-alerts')
    .on('postgres_changes', { event: '*', schema: 'public', table: 'Stock Alerts' }, () =>
    fetchData())
    .subscribe(),
    // ... more channels
  ]

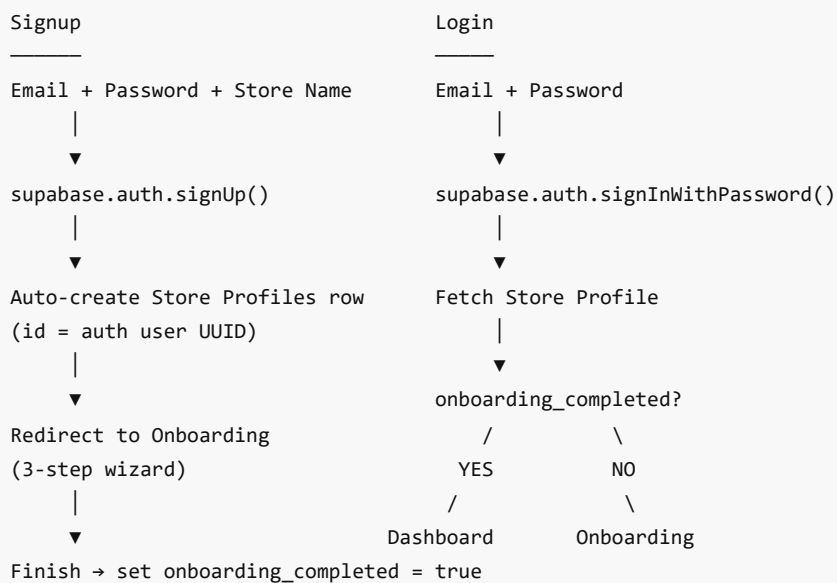
  // Cleanup on unmount
  return () => channels.forEach(c => supabase.removeChannel(c))

```

What this means in practice: When the pipeline finishes (n8n writes to Products, Alerts, etc.), the Dashboard updates **in real-time without page refresh**. This is the "Neural Pipeline" effect — the user sees the UI update live as the pipeline progresses.

12. Security & Authentication

Auth Flow



Protected Routes

```

// ProtectedRoute component wraps all authenticated pages
<ProtectedRoute>
  <Layout> ← Sidebar + Outlet
    <Dashboard />
    <Products />
    ...
  </Layout>
</ProtectedRoute>

```


If not authenticated → redirects to `/login` . If authenticated but onboarding not done → redirects to `/onboarding` .

13. Key Design Decisions & Trade-offs

Decision	Why	Trade-off
Direct Supabase access (no REST API)	Eliminates an entire backend service. Faster to build.	Tightly coupled to Supabase. Row Level Security (RLS) becomes critical.
n8n for business logic (no server code)	Visual workflows are easy to modify/debug. No deployment pipeline needed.	n8n must be running for pipeline to work. Not suitable for sub-millisecond latency.
Full-refresh alerts (delete + recreate)	Guarantees no stale alerts. Simplifies logic — no "diff" needed.	Slightly less efficient than incremental updates. Fine for < 1000 products.
Groq / Llama (not OpenAI)	Free, fast, open-source model. No vendor lock-in.	Smaller model (8B params) — less "smart" than GPT-4, but sufficient for our use case.
Frontend OOS override	Clearer UX — "OOS" is more intuitive than "HIGH" when stock=0.	Risk label in DB may say "HIGH" but UI shows "OOS". Not a bug — intentional UX decision.
Tailwind v4 (no config file)	Zero-config styling. <code>@theme</code> directive in CSS is cleaner.	Newer version — fewer community examples.

14. Demo Script — Review 2 Walkthrough

Before the Demo

- ✓ Ensure `npm run dev` is running (frontend on :5173)
- ✓ Ensure n8n is running (backend on :5678)
- ✓ Open the app in Chrome. Log in if needed.
- ✓ You should see the Dashboard with current data.

Demo Flow (15–20 minutes)

PHASE 1: The Problem Statement (2 min)

"Indian kirana stores manage hundreds of SKUs manually. They don't know when products will go out of stock, how much to reorder, or which supplier gives the best value. StockSense AI solves this."

PHASE 2: Data Input — Show CSV Import (3 min)

- Navigate to **Products** page
- Click **Import CSV** → show the template
- Explain: "Store owners upload their inventory via CSV. Each row hits our n8n webhook (WF-01) which validates, cleans, and upserts the data."
- Show the products table — point out columns: stock, threshold, risk, supplier

PHASE 3: The Intelligence Pipeline (5 min) ★

1. Go to **Dashboard**
2. Click **"Run Pipeline"** ← This is the core demo moment
3. While the pipeline runs, explain each step:
 - "Step 1: It simulates daily stock depletion based on sales velocity"
 - "Step 2: Calculates days-to-stockout for every product"
 - "Step 3: Generates stock alerts for items below threshold"
 - "Step 4: Classifies every product as HIGH / MEDIUM / LOW risk"
 - "Step 5: Sends all at-risk products to our Groq AI (Llama 3.1) which generates smart reorder recommendations"
 - "Step 6: Sends WhatsApp alerts via Twilio"
 - "Step 7: Saves a daily health snapshot for trend analysis"
4. When complete, point out:
 - **Health score** changed
 - **KPI cards** updated in real-time
 - **Needs Attention** table populated

PHASE 4: Exploring the Intelligence (5 min)

1. **Alerts page:** Show active alerts, dismiss one. Explain the stock bar visualization.
2. **Reorder page:** Show AI recommendations with the ✨ AI badge. Point out:
 - "Each recommendation includes a quantity, a matched supplier, and an AI-generated reason."
 - Click **"WhatsApp Supplier"** to show the click-to-chat integration.
3. **Suppliers page:** Show the performance chart and scoring breakdown.
 - "Suppliers are scored using a Weighted Sum Model — 40% reliability, 35% price, 25% delivery speed."

PHASE 5: Tech Architecture (3 min)

"The architecture is serverless-first. React frontend talks directly to Supabase for all data via its JavaScript SDK. Business logic runs as n8n workflows triggered by webhooks. AI runs on Groq's inference API using Llama 3.1. WhatsApp via Twilio. Everything connects via webhooks and Supabase Realtime WebSockets."

Show the architecture diagram from this document if asked.

PHASE 6: Wrap Up — Q&A Readiness

Anticipated Questions & Answers:

Question	Answer
"How does the AI work?"	"WF-05 builds a context-rich prompt with product data + supplier profiles, sends to Groq (Llama 3.1 8B), parses the JSON response into reorder recommendations."
"Why n8n instead of Express/Django?"	"n8n gives us visual workflow design, 400+ integrations, and zero deployment overhead. We can modify business logic without writing or redeploying code."
"How is data secured?"	"Supabase Auth with email/password, protected routes in React, and RLS policies on the database. Each store only sees its own data via store_id."
"Can this scale?"	"Supabase is built on PostgreSQL — handles millions of rows. n8n can be clustered. The bottleneck would be the Groq API rate limit, which is solvable with a paid tier."

"Why not use GPT-4?"	"Groq + Llama is free, faster (100ms vs 1-3s), and uses an open-source model with no vendor lock-in. For our use case (structured JSON reorder recommendations), 8B params is sufficient."
"What happens if n8n is down?"	"The frontend still works for manual operations (add/edit products, view data). Only the automated pipeline and AI features require n8n."
"How does the WhatsApp integration work?"	"Two ways: (1) WF-07 uses Twilio API to send automated alerts. (2) Reorder page uses WhatsApp Click-to-Chat links for contacting suppliers — no API needed."

 **Pro Tip:** During the demo, keep the Supabase dashboard open in another tab. If asked about the database, switch to it and show the tables with live data. This demonstrates the "real" backend.